МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №2

Тема: **«Порождающие паттерны: Абстрактная фабрика, Фабричный метод, Прототип»**

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: реализовать решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием 2 паттернов из списка: «Абстрактная фабрика», «Фабричный метод», «Прототип».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Фабричный метод

Находится в coin_collector/factory.py, классы GameObjectFactory и ArcadeObjectFactory.

Задача: отделить код сборки уровня от выбора конкретных классов игровых объектов. Строителю достаточно запросить «создать монету» или «создать опасность»; ему не нужно вручную вызывать конструкторы каждой сущности.

Как работает: абстрактная фабрика объявляет метод create(), а конкретная фабрика решает, какой экземпляр вернуть для каждого значения ObjectKind.

```
class GameObjectFactory(ABC):
    @abstractmethod
    def create(self, kind: ObjectKind, position: tuple[int, int]) -> GameObject:
        raise NotImplementedError


class ArcadeObjectFactory(GameObjectFactory):
    def create(self, kind: ObjectKind, position: tuple[int, int]) -> GameObject:
        if kind is ObjectKind.PLAYER:
            return Player(position)
        if kind is ObjectKind.COIN:
            return self._coin_prototype.clone_at(position)
        if kind is ObjectKind.HAZARD:
            return self._hazard_prototype.clone_at(position)
        raise ValueError(f"Unsupported game object kind: {kind!r}")
```

Вызов методов класса:

```
factory = ArcadeObjectFactory()

player = factory.create(ObjectKind.PLAYER, (480, 360))
coin = factory.create(ObjectKind.COIN, (120, 150))
hazard = factory.create(ObjectKind.HAZARD, (480, 220))
```

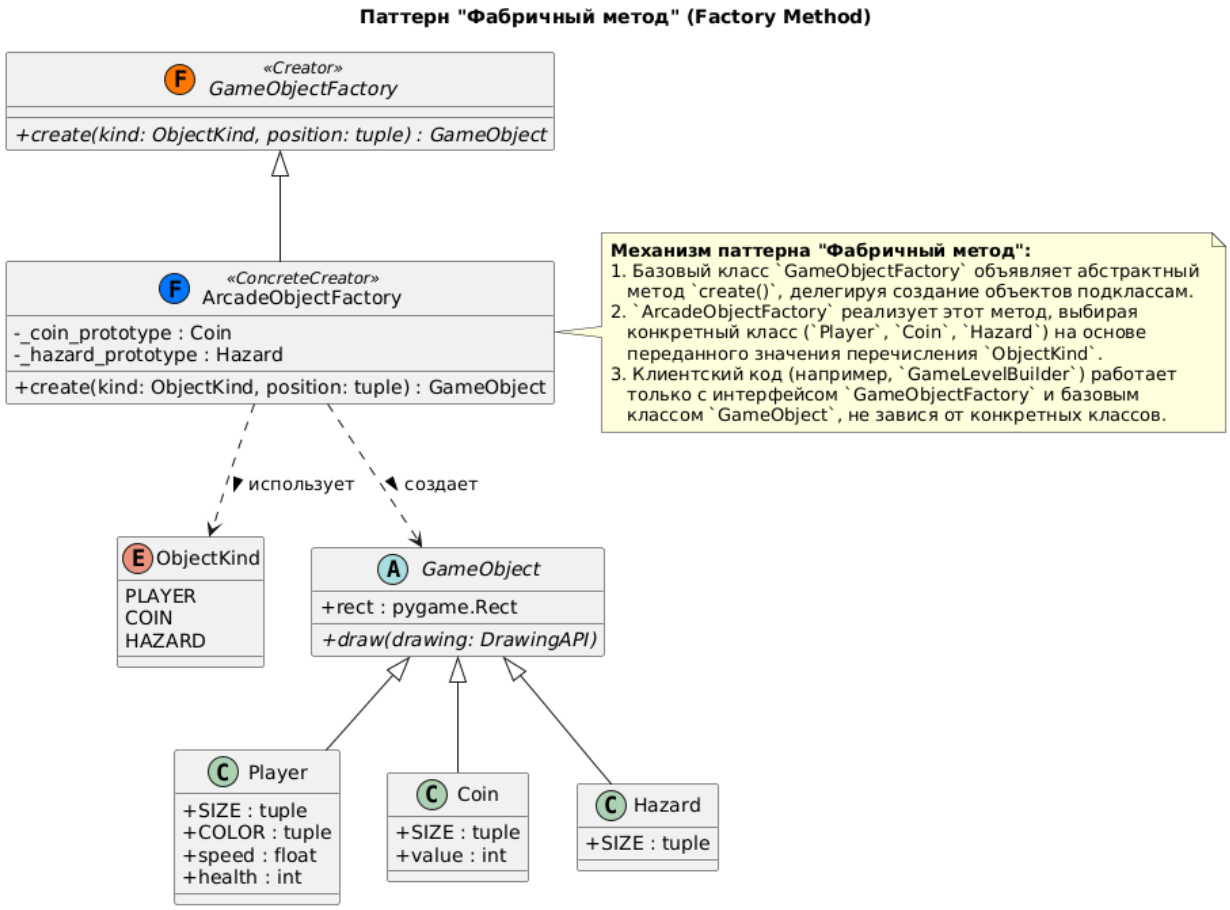


Рисунок 1. Диаграмма паттерна "Фабричный метод"

GameLevelBuilder принимает GameObjectFactory, то есть зависит от интерфейса создания, а не от конкретной фабрики. Это позволяет добавить фабрику другого режима или набора объектов без изменения последовательности шагов построения уровня. Таким образом, создание игрока, монет и препятствий сосредоточено в одном месте, а строитель уровня остаётся универсальным.

2. Прототип

Находится в coin_collector/entities.py: методы clone_at() классов Coin и Hazard; прототипы хранятся в ArcadeObjectFactory.

Задача: быстро создавать однотипные монеты и опасности с теми же начальными параметрами, но с независимыми координатами и состоянием анимации.

Как работает: clone_at() копирует объект, создаёт для копии отдельный pygame.Rect, задаёт новую позицию и сбрасывает время анимации.

```
first = factory.create(ObjectKind.COIN, (120, 150))
second = factory.create(ObjectKind.COIN, (840, 150))

assert first is not second
assert first.rect is not second.rect
assert first.rect.center == (120, 150)
assert second.rect.center == (840, 150)
```

Реализация клонирования монеты:

```
def clone_at(self, position: tuple[int, int]) -> Coin:
    clone = copy.copy(self)
    clone.rect = self.rect.copy()
    clone.rect.center = position
    clone.animation_time = 0.0
    return clone
```

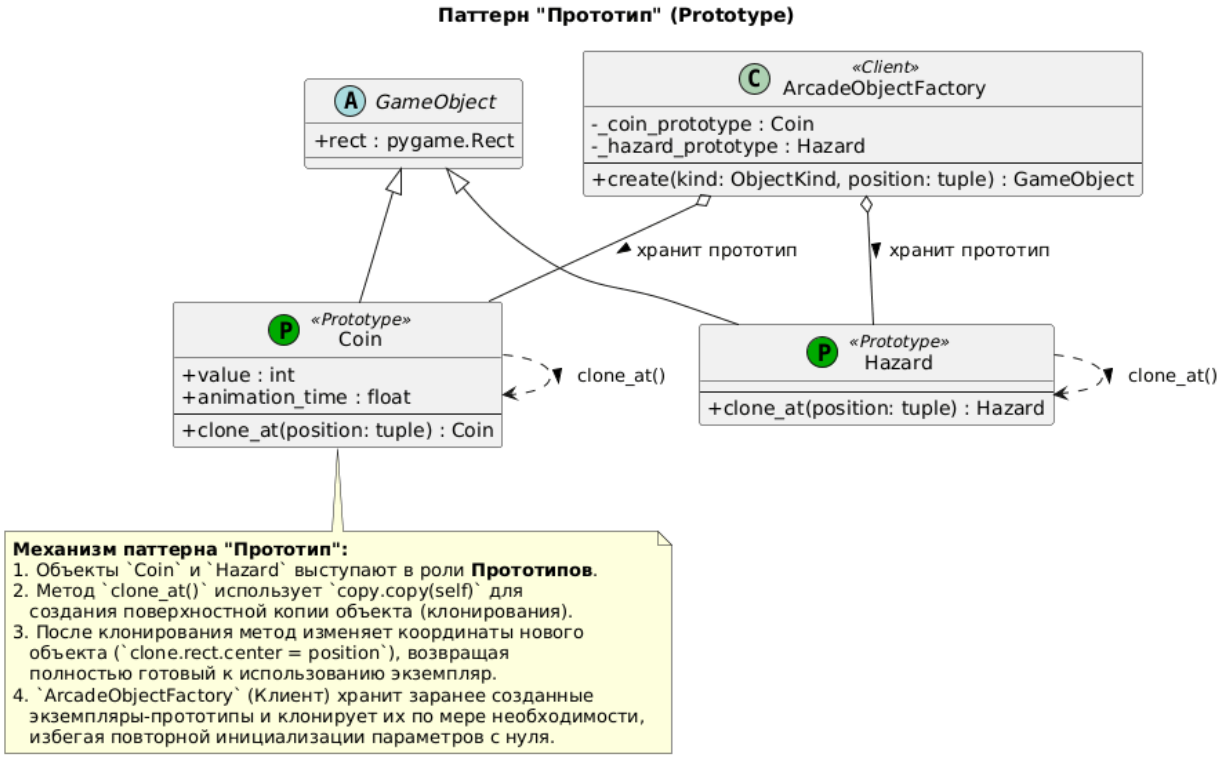


Рисунок 2. Диаграмма паттерна "Прототип"

Таким образом, на уровне появляется несколько предметов с общими базовыми параметрами, но собственными позициями и анимацией.

Вывод:

Проделав работу, реализовал решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов «Фабричный метод» и «Прототип».